

Università di Salerno

DIPARTIMENTO DI INGEGNERIA DELL'INFORMAZIONE E MATEMATICA
APPLICATA



Software Architecture Design Project Work - Gruppo 12

Nome e Cognome	Matricola	E-Mail Istituzionale
Rosa Passaro	IE22700227	r.passaro5@studenti.unisa.it
Omar Silano	IE22700120	o.silano@studenti.unisa.it
Simone Luongo	IE22700271	s.luongo10@studenti.unisa.it
Giuseppe Daniele Robinia	IE22700302	g.robinia5@studenti.unisa.it

Sommario

1. Introduzione.....	4
1.1. User Epics.....	4
2. Fase di Pre-Game.....	5
2.1. Planning.....	5
2.2. High Level Software Architecture.....	6
2.3. Wireframe dell'Interfaccia.....	8
2.4. Mockup dell'Interfaccia.....	9
2.5. Product Backlog Iniziale.....	10
2.6. Business Rules.....	23
2.7. Definition of Done.....	25
2.8. Burndown Chart.....	25
2.9. First Sprint Planning.....	26

1. Introduzione

In questo progetto, il gruppo di lavoro ha affrontato la progettazione e lo sviluppo di un'applicazione desktop per la gestione di tracce e playlist musicali, seguendo la metodologia Agile Scrum.

L'obiettivo principale del software consiste nel consentire all'utente di inserire i brani che preferisce e creare e gestire playlist.

Partendo dalle *User Epic* fornite come specifica iniziale, sono state definite le relative *User Story* ed è stato organizzato lo sviluppo in tre Sprint consecutivi.

Il gruppo ha configurato un *repository* Git su GitHub per centralizzare la gestione del codice sorgente e dei relativi elementi che compongono il progetto.

Il repository contiene:

- Il codice sorgente sviluppato in Java.
- Gli unit test automatizzati basati su JUnit.
- Un documento descrittivo dell'architettura software.
- Il Product Backlog e gli Sprint Backlog per ciascuna iterazione.
- La presentazione e i report conclusivi.

1.1. User Epics

Di seguito vengono descritte le User Epics in ordine di priorità.

1.1.1. Priorità alta:

- L'applicazione deve permettere di gestire (creare, modificare, eliminare e mostrare) tracce (canzoni) e organizzare (aggiungendo e rimuovendo) le tracce nelle playlist. Le tracce sono rappresentate con un titolo, l'autore, la durata, il genere e l'anno di pubblicazione. Le playlist sono identificate da un nome. L'intera playlist e le singole tracce possono essere riprodotte, messe in pausa o saltate.
- La riproduzione di tracce o playlist può avvenire in tre diverse modalità: sequenziale, casuale, ripetizione (loop). La playlist può essere modificata (aggiungendo o rimuovendo tracce) in qualsiasi momento, anche durante la riproduzione.
- Le tracce e le playlist sono visualizzate nell'interfaccia utente del lettore multimediale e si aggiornano in tempo reale con l'avanzamento della riproduzione.

1.1.2. Priorità media

- Qualsiasi operazione di aggiunta o rimozione deve essere revocabile (Undo). L'applicazione deve mostrare nella pagina principale i brani e le playlist più ascoltati. L'applicazione deve permettere la creazione automatica di playlist basate sul genere o sull'anno.
- A ogni traccia dovrebbe essere possibile aggiungere tag visivi (preferito, esplicito, nuova uscita) per arricchire il modo in cui vengono visualizzate e usarli per la creazione automatica delle playlist.

1.1.3. Priorità bassa

- Le tracce in una playlist dovrebbero essere riordinate in qualsiasi momento, anche durante la riproduzione. (possibile observer?)

2. Fase di Pre-Game

In questa fase viene pianificato l'avvio del progetto, definendo la strumentazione che verrà impiegata, gli strumenti di supporto, l'architettura ad alto livello del prodotto da sviluppare e il Product Backlog contenente le User Story da implementare. Vengono inoltre presentate le convenzioni adottate e alcuni parametri caratteristici del framework Scrum.

2.1. Planning

Per lo sviluppo del progetto saranno impiegate le seguenti tecnologie:

- **Java JDK 21:** linguaggio di programmazione impiegato per lo sviluppo di applicazioni orientate agli oggetti (Object-Oriented).
- **JavaFX versione 21.0.1:** librerie esterne utilizzate per lo sviluppo di interfacce grafiche in combinazione con il linguaggio Java.
- **JUnit:** framework utilizzato per l'esecuzione di test automatizzati su applicazioni sviluppate in linguaggio Java.

Gli strumenti ausiliari adottati per il progetto includono:

- **GitHub:** piattaforma impiegata per agevolare la condivisione del codice sorgente e della documentazione tecnica tra i membri del gruppo.
Link: https://github.com/ciiprosk/MusicPlaylistManager_SAD.git
- **Trello:** piattaforma gestionale adottata per monitorare agevolmente l'avanzamento e lo stato del progetto ed elaborazione di grafici.
Link: <https://trello.com/b/f0J0mS0W/project-work-sad>

Le convenzioni adottate sono le seguenti:

- **Notazione CamelCase:** standard adottato per la nomenclatura di variabili e funzioni all'interno del codice sorgente.
- **Lingua inglese:** lingua impiegata per la stesura del codice sorgente;
- **Lingua italiana:** lingua impiegata per scrivere commenti e documentazione tecnica.
- **Sequenza di Fibonacci:** utilizzata per la stima e l'assegnazione degli Story Point. Verranno considerati esclusivamente i primi cinque valori della sequenza (1, 2, 3, 5, 8).
- **Livelli di priorità:** la priorità è strutturata su tre livelli: Alta, Media, Bassa. All'interno della piattaforma Trello, tali livelli corrispondono alle etichette cromatiche rossa, arancione e verde.

2.2. High Level Software Architecture

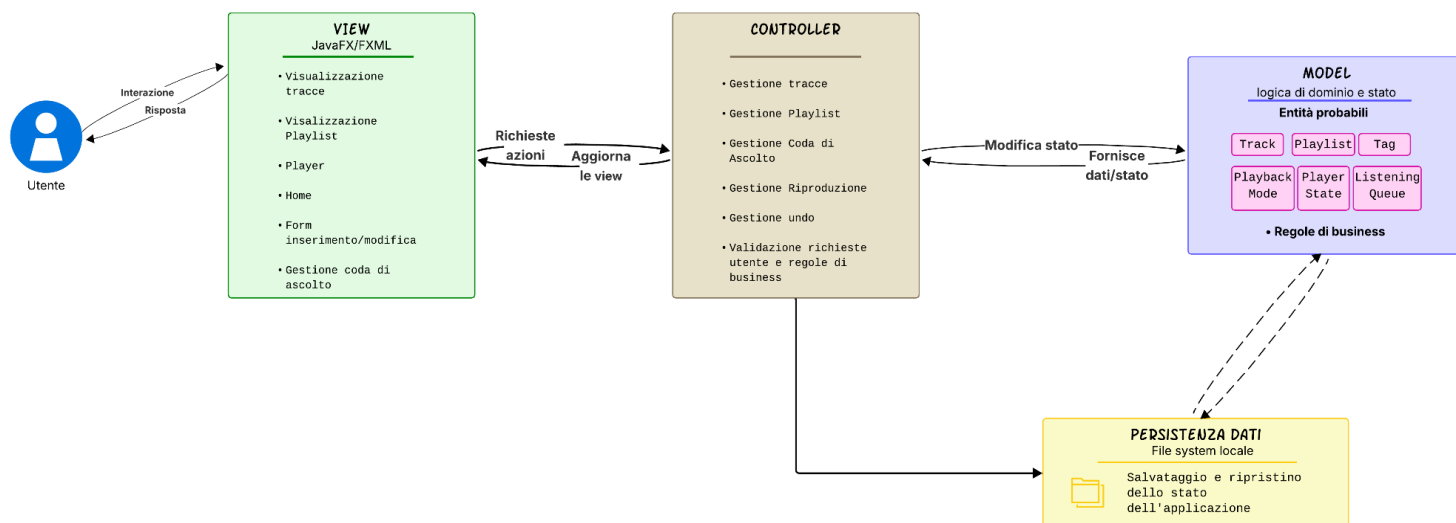
La progettazione del software (Software Design) si occupa in primo luogo di definire la struttura del sistema, ovvero di identificare i componenti fondamentali e le relazioni che intercorrono tra di essi. La fase di progettazione architetturale fornisce una visione ad alto livello (macro-view) dell'intero sistema.

L'architettura software di un programma è definita dalla struttura (o dalle strutture) del sistema stesso, la quale comprende gli elementi software, le proprietà esternamente visibili di tali elementi e le loro reciproche relazioni. Molte architetture di sistema adottano uno stile riconoscibile. In particolare, l'architettura scelta per lo sviluppo di questa applicazione segue il pattern architetturale **Model-View-Controller (MVC)**.

- **Model (Modello):** Rappresenta il livello in cui è incapsulata la logica di dominio (core logic) e la gestione dei dati dell'applicazione. In questo progetto, il Model gestisce la struttura dell'archivio musicale, le entità (come *Track* e *Playlist*), la validazione delle Business Rules e la manipolazione della coda di ascolto.
- **View (Vista):** Rappresentata in questo contesto dai documenti *FXML* di JavaFX, la View gestisce l'interfaccia grafica utente (GUI). Il suo compito è mostrare i dati all'utente e catturare gli input (come i click sui controlli del player), senza tuttavia elaborarli o manipolarli direttamente. Gli input vengono invece inoltrati passivamente al Controller.
- **Controller (Controllore):** Agisce come intermediario diretto tra la View e il Model. Riceve le azioni dall'interfaccia, le interpreta (eseguendo la logica applicativa necessaria) e aggiorna il Model di conseguenza. Inoltre, il Controller si occupa di aggiornare dinamicamente la View in risposta ai cambiamenti di stato del Model (come l'avanzamento della barra del player in tempo reale), garantendo la costante sincronizzazione tra l'interfaccia e i dati.

Il flusso di interazione standard prevede che l'utente interagisca con la View, la quale inoltra le richieste al Controller; quest'ultimo elabora l'input, altera lo stato del Model e innesca il refresh della View. Questa netta separazione delle responsabilità (*Separation of Concerns*) incrementa significativamente la modularità del sistema, semplifica le operazioni di manutenzione e migliora la testabilità complessiva del software, facilitando l'implementazione degli unit test automatizzati.

Di seguito è riportato il diagramma che illustra l'architettura ad alto livello adottata per il sistema:

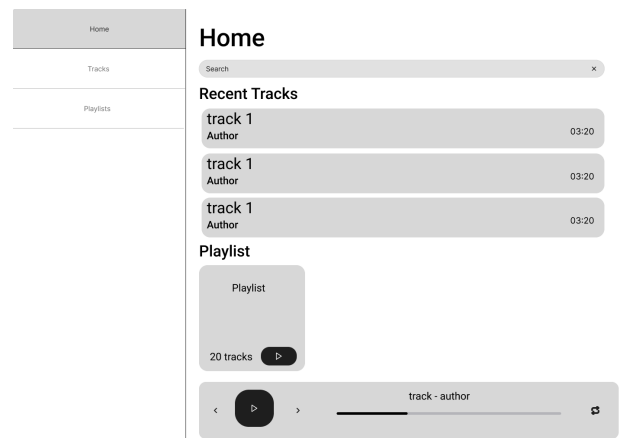
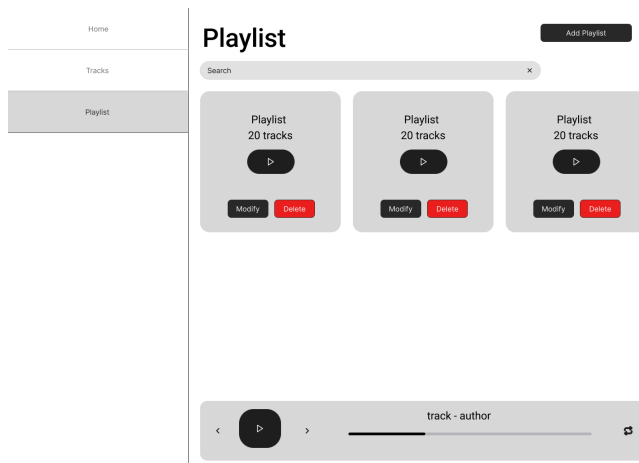
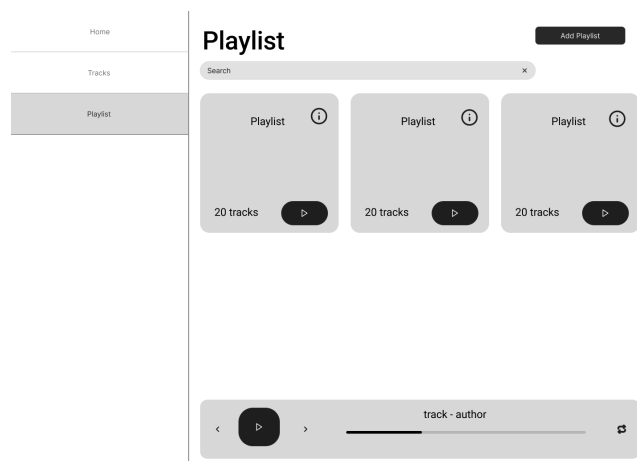
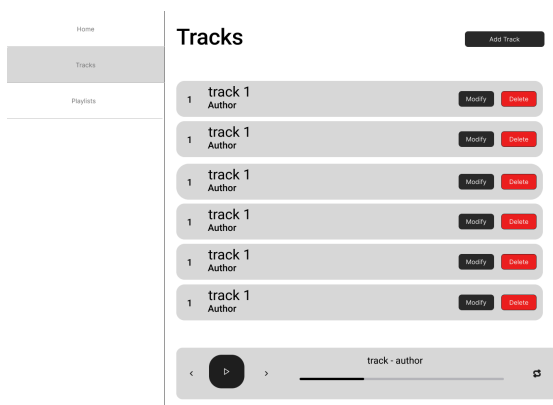


A supporto della logica di dominio gestita dal Model, l'architettura integra un fondamentale meccanismo di **Persistenza dei Dati**. Poiché le informazioni (come le code di riproduzione e gli oggetti istanziati) risiedono in memoria volatile durante l'esecuzione dell'applicazione, è strettamente necessario garantire che l'archivio musicale, le playlist create dall'utente e i metadati associati non vadano persi alla chiusura del programma. A tale scopo, il sistema è progettato per serializzare e consolidare lo stato corrente dell'applicazione su un supporto di memorizzazione persistente (ad esempio tramite file system locale in formato JSON o simili). Al successivo avvio, l'applicazione si occupa di leggere e decodificare tali dati, ricostruendo istantaneamente il grafo degli oggetti nel Model e ripristinando l'intero contesto operativo, assicurando così l'integrità dei dati e la continuità dell'esperienza di utilizzo tra sessioni diverse.

2.3. Wireframe dell'Interfaccia

Prima di definire l'aspetto grafico definitivo, il team ha sviluppato un set di wireframe a bassa fedeltà (*Low-Fidelity*).

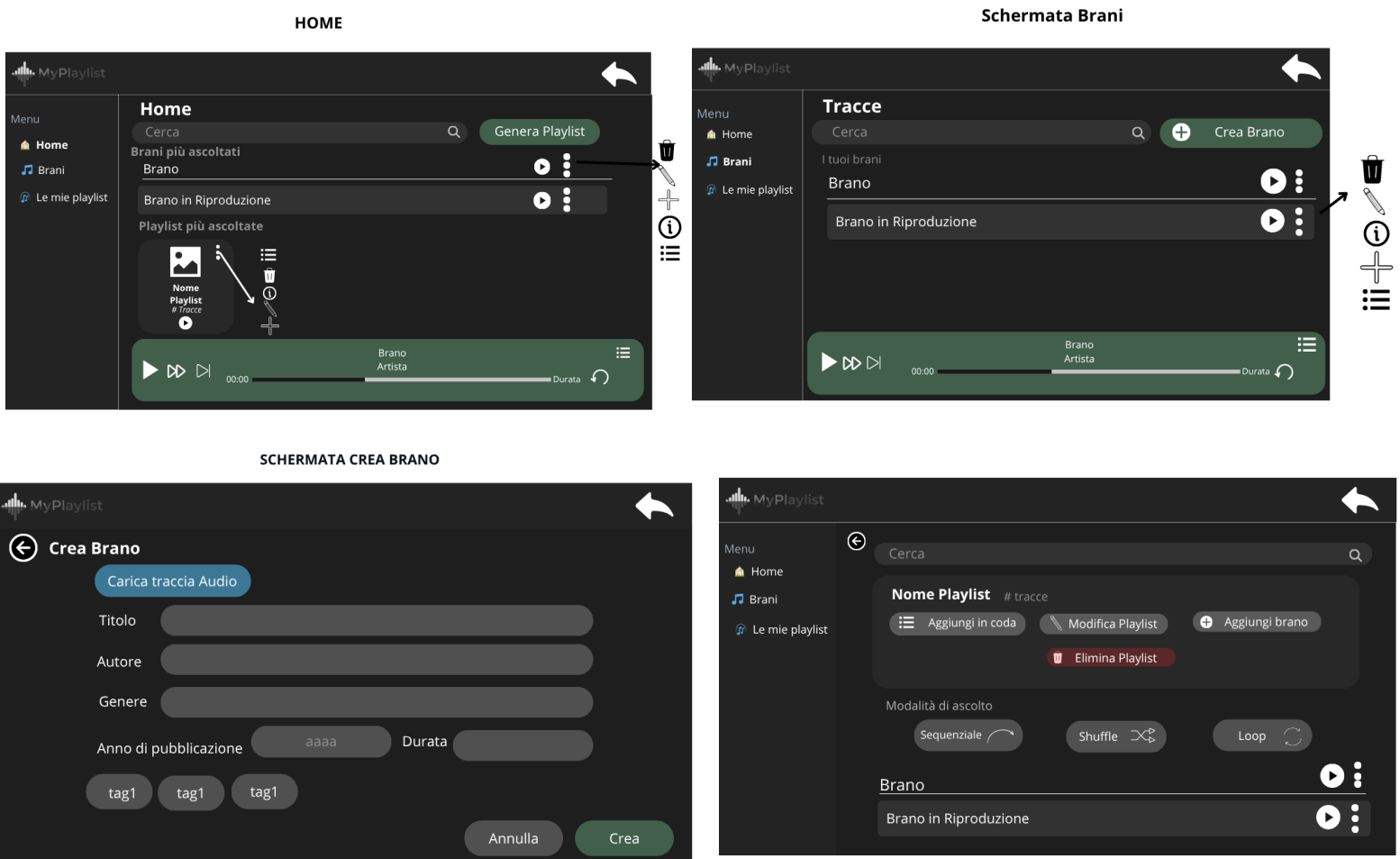
L'obiettivo di questa fase preliminare è stato quello di definire la gerarchia visiva delle informazioni e l'architettura dei contenuti, assicurandosi che le funzionalità descritte nelle User Story (come la gestione delle code di riproduzione e l'interazione con le playlist) fossero facilmente accessibili. L'assenza intenzionale di colori e dettagli stilistici in questa fase ha permesso al gruppo di concentrarsi esclusivamente sull'usabilità, sull'ingombro dei componenti (layout strutturale) e sul flusso di navigazione dell'utente, validando i requisiti funzionali prima di procedere all'implementazione.



2.4.Mockup dell'Interfaccia

A valle della validazione strutturale ottenuta tramite i wireframe, il team ha realizzato i mockup ad alta fedeltà (High-Fidelity) dell'applicazione. Lo scopo di questo artefatto è definire in modo preciso l'aspetto visivo e la palette cromatica prima di procedere all'implementazione delle viste tramite *JavaFX* e *Scene Builder*.

Le immagini seguenti costituiscono il riferimento visivo e funzionale per lo sviluppo del livello *View* all'interno del pattern architetturale del sistema.



2.5. Product Backlog Iniziale

Di seguito saranno elencate le user epics con le relative user stories identificate, e la velocità ideale che sarà considerata per lo sviluppo dell'applicazione in 3 Sprint.

La stima degli sforzi è stata condotta in modo collaborativo da tutti i membri del team tramite la tecnica del *Planning Poker*, utilizzando i mockup come supporto visivo per valutare la complessità delle interfacce e delle interazioni. In questo contesto, gli story points sono stati utilizzati come misura relativa della quantità e della complessità del lavoro, combinando difficoltà e tempo necessari per la realizzazione. È stato importante non utilizzare valori numerati in modo strettamente sequenziale, perché la complessità non cresce in modo lineare: la differenza tra valori piccoli è più significativa rispetto a quella tra valori grandi. Per questo motivo, come definito nelle convenzioni di progetto, per l'assegnazione degli Story Points è stata utilizzata la sequenza di Fibonacci (1, 2, 3, 5, 8). Questo approccio ha permesso di ottenere stime relative coerenti con il framework Scrum, escludendo l'impiego di modelli algoritmici tradizionali.

Il carico di lavoro complessivo stimato per l'intero Product Backlog ammonta a **81 Story Points**. Considerando che lo sviluppo dell'applicazione è pianificato su 3 Sprint, la stima di velocità del team è di **27 Story Points per Sprint**. Tale valore costituisce la metrica di riferimento fondamentale per la distribuzione del carico di lavoro nei successivi Sprint Backlog.

Tutti i membri del gruppo rivestono il ruolo di Product Owner.

Nella definizione delle User Stories è stato seguito il modello INVEST, con l'accortezza di controllare che fossero indipendenti tra loro per favorire il lavoro in parallelo.

Inoltre, le storie sono state scritte in modo che siano stimabili in termini di complessità, sufficientemente piccole da poter essere completate all'interno di uno sprint e verificabili tramite criteri di accettazione e test chiari.

Inoltre sono stati definiti criteri di accettazione al fine di concordare una visione comune delle possibili problematiche tra tutti i membri del team e soprattutto a definire quando un'attività può essere considerata 'Done'.

2.5.1. User Stories sulla Gestione delle Tracce

- **US1 - Creazione Tracce**

- **Story Points:** 5
- **Priorità:** Alta
- **Descrizione:**
 - Come utente,*
voglio inserire tracce specificando titolo, autore, genere e anno di
pubblicazione,
per aggiungerli nell'archivio musicale.
- **Criteri di Accettazione:**
 1. *Dato che mi trovo nel form di aggiunta,*
quando compilo tutti i campi con un formato valido e confermo,

allora il sistema salva la traccia e questa appare nell'archivio con i relativi dati.

2. *Dato che mi trovo nel form di aggiunta,
quando tento di salvare con un formato errato almeno uno dei campi,
allora il sistema blocca il salvataggio e viene visualizzato un messaggio di errore.*
3. *Dato che mi trovo nel form di aggiunta,
quando inserisco i campi in modo identico a un'altra traccia,
allora il sistema non permette il salvataggio del brano e avvisa l'utente con un messaggio.*

- **US2 - Modifica Tracce**

- **Story Points:** 3

- **Priorità:** Alta

- **Descrizione:**

- Come utente,
voglio modificare le tracce,
così da cambiare le informazioni inserite in caso di errore.*

- **Criteri di Accettazione:**

- 1. *Dato che sono nel form di modifica di una traccia già presente in archivio, quando modifico tutti i campi con un formato valido e confermo, allora la traccia appare modificata nell'archivio con i relativi dati.*
 - 2. *Dato che sono nel form di modifica di una traccia già presente in archivio, quando modifico tutti i campi con un formato non valido, allora il bottone di conferma viene disattivato e appare un messaggio informativo nei campi interessati.*
 - 3. *Dato che sono nel form di modifica di una traccia già presente in archivio, quando premo il tasto per annullare l'operazione, allora il form termina e i dati originali della traccia rimangono inalterati.*

- **US3 - Eliminazione Tracce**

- **Story Points:** 2

- **Priorità:** Alta

- **Descrizione:**

- Come utente,
voglio eliminare tracce,
così da non avere più sul programma tracce inserite erroneamente o non più desiderate.*

- **Criteri di Accettazione:**

- 1. *Dato che una traccia è presente e selezionata nell'archivio, quando premo il tasto per la cancellazione e confermo, allora la canzone viene rimossa definitivamente dal sistema e scompare dall'elenco visivo.*
 - 2. *Dato che ho premuto il pulsante di eliminazione, quando appare l'avviso di conferma e scelgo "Annulla", allora l'operazione viene interrotta e la traccia rimane nell'archivio.*

- **US4 - Visualizzazione Tracce**

- **Story Points:**1

- **Priorità:** Alta

- **Descrizione:**

*Come utente,
voglio visualizzare tutte le tracce,
così da poter leggere le informazioni associate e poterle organizzare in playlist
e ascoltarle.*

- **Criteri di Accettazione:**

1. *Dato che l'archivio musicale contiene almeno una traccia salvata,
quando accedo alla schermata apposita,
allora visualizzo l'elenco di tutte le tracce presenti con i relativi metadati
visibili (titolo, autore, durata).*
2. *Dato che l'archivio musicale è vuoto (zero tracce salvate),
quando accedo alla schermata apposita,
allora visualizzo un messaggio informativo ("Vuoto" o invito ad aggiungere
nuova traccia).*

- **US5 - Assegnazione Tag alle Tracce**

- **Story Points:** 3

- **Priorità:** Alta

- **Descrizione:**

*Come utente,
voglio aggiungere tag visivi (preferito, esplicito, nuova uscita),
perché mi facilita la visualizzazione delle tracce.*

- **Criteri di Accettazione:**

1. *Dato che visualizzo un brano,
quando aggiungo uno o più tag e salvo,
allora i tag sono associati al brano e visibili nel dettaglio.*

2.5.2. User Stories sulla Gestione delle Playlist

- **US6 - Creazione Playlist**

- **Story Points:** 2
- **Priorità:** Alta
- **Descrizione:**

*Come utente,
voglio creare una playlist identificata da un nome univoco,
per organizzare i brani presenti nell'archivio secondo un determinato criterio.*
- **Criteri di Accettazione:**
 1. *Dato che accedo alla sezione di creazione delle playlist,
quando inserisco un nome univoco alla nuova playlist e confermo,
allora la playlist viene creata vuota ed è visibile nella lista.*
 2. *Dato che accedo alla sezione di creazione delle playlist,
quando inserisco un nome alla nuova playlist,
allora il sistema vieta la creazione della nuova playlist e restituisce un
messaggio di errore.*

- **US7 - Aggiunta Tracce a Playlist**

- **Story Points:** 2
- **Priorità:** Alta
- **Descrizione:**

*Come utente,
voglio aggiungere tracce ad una playlist anche se in riproduzione,
al fine di poterla ascoltare insieme alle altre tracce della playlist.*
- **Criteri di Accettazione:**
 1. *Dato che visualizzo una playlist,
quando aggiungo un brano dall'archivio,
allora il brano appare in fondo alla playlist.*
 2. *Dato che visualizzo una playlist,
quando tento di aggiungere un brano già presente in essa,
allora il sistema non permette l'inserimento del duplicato specificandolo
all'utente attraverso un messaggio.*

- **US8 - Rimozione Tracce da Playlist**

- **Story Points:** 1
- **Priorità:** Alta
- **Descrizione:**

*Come utente,
voglio eliminare tracce da una playlist anche se in riproduzione,*

al fine di visualizzare solo le tracce che voglio nella playlist.

- **Criteri di Accettazione:**

1. *Dato che visualizzo una playlist,
quando rimuovo una traccia e confermo,
allora la traccia non è più presente nella playlist ma rimane nell'archivio.*

- **US9 - Visualizzazione Playlist**

- **Story Points:** 1

- **Priorità:** Alta

- **Descrizione:**

*Come utente,
voglio visualizzare una playlist,
in modo da poterne consultare il contenuto.*

- **Criteri di Accettazione:**

1. *Dato che accedo a una playlist,
quando si apre,
allora sono visibili nome, elenco delle tracce in ordine di aggiunta con titolo e autore.*

- **US10 - Modifica Nome Playlist**

- **Story Points:** 1

- **Priorità:** Alta

- **Descrizione:**

*Come utente,
voglio poter modificare il nome di una playlist,
per correggere eventuali errori o aggiornare le informazioni associate.*

- **Criteri di Accettazione:**

1. *Dato che sto modificando il nome di una playlist esistente,
quando inserisco il nuovo nome con un formato valido e confermo,
allora la playlist appare aggiornata nell'elenco con le nuove informazioni.*

- **US11 - Generazione Playlist per Genere**

- **Story Points:** 3

- **Priorità:** Media

- **Descrizione:**

*Come utente,
voglio generare playlist filtrate per genere,
al fine di organizzare automaticamente la mia musica.*

- **Criteri di Accettazione:**

1. *Dato che seleziono un genere,
quando genero una nuova playlist automatica tramite genere,*

allora il sistema crea una playlist contenente tracce di quel genere.

- **US12 - Generazione Playlist per Anno**

- **Story Points:** 3
- **Priorità:** Media
- **Descrizione:**

*Come utente,
voglio generare playlist filtrate per anno di pubblicazione,
al fine di organizzare automaticamente la mia musica.*
- **Criteri di Accettazione:**
 1. *Dato che seleziono un anno,
quando genero una nuova playlist automatica tramite l'anno di uscita,
allora il sistema crea una playlist contenente tracce uscite in quell'anno.*

- **US13 - Generazione Playlist Tramite Tag**

- **Story Points:** 3
- **Priorità:** Media
- **Descrizione:**

*Come utente,
voglio utilizzare i tag per generare playlist automatiche,
per filtrare e organizzare la musica in modo intelligente.*
- **Criteri di Accettazione:**
 1. *Dato che seleziono un tag,
quando genero una nuova playlist automatica tramite i tag,
allora il sistema crea una playlist contenente tracce con quel tag.*

- **US14 - Riordinamento Tracce in Playlist**

- **Story Points:** 3
- **Priorità:** Bassa
- **Descrizione:**

*Come utente,
voglio poter riordinare le tracce in una playlist,
perché posso avere la necessità di dover fare dei cambiamenti di ordine.*
- **Criteri di Accettazione:**
 1. *Dato che visualizzo il contenuto di una playlist,
quando sposto una traccia in una posizione diversa da quella originale,
allora il nuovo ordine viene salvato e la lista si aggiorna immediatamente mostrandolo.*
 2. *Dato che l'ordine delle tracce è stato cambiato,
quando inizia una nuova riproduzione,
allora la riproduzione in sequenza prevede il nuovo ordine creato.*

- **US15 - Riordinamento Dinamico durante la Riproduzione**

- **Story Points:** 5
- **Priorità:** Bassa
- **Descrizione:**

*Come utente,
voglio poter riordinare le tracce in una playlist durante la riproduzione di una traccia,
perché mentre ascolto le tracce posso avere la necessità di dover fare dei cambiamenti di ordine.*
- **Criteri di Accettazione:**
 1. *Dato che una traccia è in riproduzione,
quando modifico l'ordine delle tracce in una playlist,
allora la riproduzione della traccia non viene interrotta.*
 2. *Dato che l'ordine delle tracce è stato cambiato,
quando è terminata la traccia corrente,
allora la riproduzione in sequenza prevede il nuovo ordine creato.*

2.5.3. User Stories sulla Gestione della Coda di Ascolto

- **US16 - Riproduzione Traccia**

- **Story Points:** 2
- **Priorità:** Alta
- **Descrizione:**

*Come utente,
voglio riprodurre una traccia,
per ascoltare la musica.*
- **Criteri di Accettazione:**
 1. *Dato che mi trovo nella schermata dell'archivio musicale o all'interno di una playlist e ho selezionato una traccia,
quando premo il pulsante di riproduzione,
allora inizia la riproduzione audio del brano selezionato.*

- **US17 - Riproduzione in Pausa**

- **Story Points:** 2
- **Priorità:** Alta
- **Descrizione:**

*Come utente,
voglio mettere in pausa la riproduzione di una traccia,
per interrompere temporaneamente l'ascolto.*
- **Criteri di Accettazione:**
 1. *Dato che la riproduzione è in stato di pausa,
quando premo di nuovo il pulsante della riproduzione,*

- allora l'audio riparte esattamente dal punto in cui era stato interrotto.*
2. *Dato che un brano è in corso di riproduzione,
quando premo il pulsante di pausa,
allora l'audio si interrompe mantenendo il timer fermo nella posizione attuale.*

● US18 - Riproduzione della Traccia Successiva

- **Story Points:** 2
- **Priorità:** Alta
- **Descrizione:**

*Come utente,
voglio passare al brano successivo,
per cambiare ciò che sto ascoltando.*
- **Criteri di Accettazione:**
 1. *Dato che sto ascoltando un brano che non è l'ultimo della coda,
quando premo il tasto per passare al brano successivo,
allora la riproduzione corrente si interrompe e inizia immediatamente quella del brano seguente.*
 2. *Dato che sto ascoltando l'ultimo brano della coda,
quando premo il tasto per passare al brano successivo,
allora la riproduzione si ferma.*
 3. *Dato che la coda di ascolto è completamente vuota,
quando voglio premere il tasto per andare avanti nell'ascolto,
il sistema disabilita il tasto per andare avanti.*
 4. *Dato che una playlist è stata inserita nella coda ed è in riproduzione,
quando premo il pulsante "Salta Playlist",
allora il sistema rimuove dalla coda tutti i brani successivi appartenenti a quella playlist e passa al primo brano di origine diversa.*

● US19 - Riproduzione Sequenziale della Coda

- **Story Points:** 2
- **Priorità:** Alta
- **Descrizione:**

*Come utente,
voglio riprodurre in modalità sequenziale il contenuto della mia coda di ascolto,
per ascoltare i brani nell'ordine in cui li ho inseriti o in cui erano disposti nella playlist.*
- **Criteri di Accettazione:**
 1. *Dato che la riproduzione è in modalità Sequenziale, quando un brano termina,
allora il sistema riproduce automaticamente il brano successivo in coda,
fermandosi se è l'ultimo.*

- **US20 - Riproduzione in Loop della Coda**

- **Story Points:** 2
- **Priorità:** Alta
- **Descrizione:**

*Come utente,
voglio riprodurre in loop la coda di ascolto,
per riascoltare automaticamente la musica quando la lista finisce.*
- **Criteri di Accettazione:**
 1. *Dato che la riproduzione è in modalità Loop, quando termina l'ultimo brano presente in coda, allora la riproduzione ricomincia automaticamente dal primo brano della coda stessa.*

- **US21 - Riproduzione Shuffle della Coda**

- **Story Points:** 3
- **Priorità:** Alta
- **Descrizione:**

*Come utente,
voglio poter selezionare la modalità di riproduzione Shuffle,
per la coda di ascolto, per ottenere una riproduzione variata rimescolando
l'ordine dei brani.*
- **Criteri di Accettazione:**
 1. *Dato che la modalità Shuffle è attiva, quando un brano termina o premo "Successivo", allora il sistema seleziona casualmente il prossimo brano tra quelli rimanenti in coda.*
 2. *Dato che la modalità Shuffle è attiva, il sistema deve garantire che nessun brano venga riprodotto due volte prima che tutti i brani della coda siano stati riprodotti.*

- **US22 - Inserimento tracce nella Coda di Ascolto**

- **Story Points:** 5
- **Priorità:** Alta
- **Descrizione:**

*Come utente,
voglio inserire singole tracce o intere playlist all'interno della coda di
riproduzione,
per definire la lista dei brani che ascolterò.*
- **Criteri di Accettazione:**
 1. *Dato che sto esplorando l'archivio musicale,
quando seleziono una singola traccia e la aggiungo alla coda,
allora il brano viene accodato in fondo alla coda.*
 2. *Dato che visualizzo l'elenco delle mie playlist,
quando seleziono una playlist e scelgo di accodarla,*

allora tutte le tracce che la compongono vengono estratte e aggiunte sequenzialmente in fondo alla coda di riproduzione.

2.5.4. User Stories sull'Interfaccia Utente

- **US23 - Aggiornamento dello Stato dell'Interfaccia Utente**

- **Story Points:** 3
- **Priorità:** Alta
- **Descrizione:**

*Come utente,
voglio che l'interfaccia si aggiorni in tempo reale durante la riproduzione,
per visualizzare sempre lo stato corrente.*
- **Criteri di Accettazione:**
 1. *Dato che un brano è in riproduzione,
quando trascorre almeno un secondo,
allora il player mostra avanzamento aggiornato senza ricaricare la pagina.*

- **US24 - Visualizzazione Tracce Più Ascoltate**

- **Story Points:** 3
- **Priorità:** Media
- **Descrizione:**

*Come utente,
voglio visualizzare le 5 tracce più ascoltate nella home page,
per avere rapido accesso ai miei brani preferiti.*
- **Criteri di Accettazione:**
 1. *Dato che ho ascoltato almeno una traccia,
quando accedo alla home,
allora è visibile una sezione con i brani ordinati per numero di riproduzioni
decrescenti.*

- **US25 - Visualizzazione Playlist Più Ascoltate**

- **Story Points:** 3
- **Priorità:** Media
- **Descrizione:**

*Come utente,
voglio visualizzare le 5 playlist più ascoltate nella home page,
per scoprire i contenuti che ascolto di più.*
- **Criteri di Accettazione:**
 1. *Dato che l'applicazione ha registrato i miei ascolti alle playlist più frequenti,
quando accedo alla home,
allora sono visibili le playlist più ascoltate.*

- **US26 - Revocabilità di Aggiunta o Rimozione di un Brano dalla Playlist**

- **Story Points:** 3
- **Priorità:** Media
- **Descrizione:**

*Come utente,
voglio poter revocare l'aggiunta o la rimozione di un brano da una playlist,
per rimediare rapidamente ad un errore accidentale e ripristinare la coda desiderata.*
- **Criteri di Accettazione:**
 1. *Dato che ho appena aggiunto un brano a una playlist,
quando eseguo la revoca,
allora il brano viene rimosso dalla playlist e la lista torna allo stato precedente.*
 2. *Dato che ho appena rimosso un brano da una playlist,
quando eseguo la revoca,
allora il brano viene reinserito nella playlist nella posizione originale.*

- **US27 - Revocabilità di Aggiunta o Rimozione di una Traccia dall'Archivio**

- **Story Points:** 5
- **Priorità:** Media
- **Descrizione:**

*Come utente,
voglio poter revocare l'aggiunta o l'eliminazione di una traccia dall'archivio,
per rimediare rapidamente ad un errore accidentale e ripristinare l'archivio musicale.*
- **Criteri di Accettazione:**
 1. *Dato che ho appena aggiunto una traccia dall'archivio,
quando eseguo la revoca,
allora la traccia viene rimossa dall'archivio.*
 2. *Dato che ho appena eliminato una traccia dall'archivio,
quando eseguo la revoca,
allora la traccia viene ripristinata nell'archivio con tutti i suoi dati originali.*

- **US28 - Revocabilità di Creazione o Eliminazione di una Playlist**

- **Story Points:** 8
- **Priorità:** Media
- **Descrizione:**

*Come utente,
voglio revocare la creazione o l'eliminazione di una playlist,
per correggere operazioni fatte per sbaglio.*

- **Criteri di Accettazione:**

1. *Dato che ho appena creato una playlist, quando eseguo la revoca, allora la playlist viene eliminata e non è più visibile nella lista.*
2. *Dato che ho appena eliminato una playlist, quando eseguo la revoca, allora la playlist viene ripristinata con il nome originale e tutti i brani associati nella posizione originale.*

2.6. Business Rules

Le *Business Rules* definiscono i vincoli di validità dei dati e le logiche di dominio che l'architettura (nello specifico, il livello Model e il Controller) deve far rispettare rigorosamente,

Regole per l'entità "Traccia" (Track)

- **Identificatore (ID):** Generato automaticamente dal sistema alla creazione tramite UUID (Universally Unique Identifier). Immutabile e invisibile all'utente.
- **Titolo:** Stringa alfanumerica obbligatoria. Lunghezza massima di 100 caratteri. Non può essere nulla, vuota o composta esclusivamente da spazi bianchi.
- **Autore:** Stringa alfanumerica. Lunghezza massima di 100 caratteri. Di default, corrisponde a 'Sconosciuto'.
- **Durata:** Valore numerico intero obbligatorio, espresso internamente in secondi. Deve essere strettamente maggiore di zero (> 0). Valori negativi o formati testuali non sono ammessi. La durata viene inserita automaticamente dal sistema, e l'utente non può inserirla o modificarla.
- **Anno di Pubblicazione:** Valore numerico intero di esattamente 4 cifre. Non può essere un anno futuro. Di default, corrisponde a 'Sconosciuto'.
- **Genere:** Stringa di testo. Lunghezza massima di 50 caratteri. Di default, corrisponde a 'Sconosciuto'.
- **Regola di Duplicazione:** Il sistema considera "duplicata" una traccia se presenta l'esatta combinazione di *Titolo* e *Autore* di un brano già registrato nell'archivio.

Regole per l'entità "Playlist"

- **Identificatore (ID):** Generato automaticamente dal sistema tramite UUID. Immutabile.
- **Nome:** Stringa di testo obbligatoria. Lunghezza massima di 50 caratteri. Non può iniziare o terminare con un carattere speciale. Non può essere composta solo da spazi.
- **Unicità del Nome:** Non è possibile avere due playlist con lo stesso nome.
- **Contenuto:** Una playlist memorizza unicamente i riferimenti (ID) alle tracce. Può esistere, anche se vuota (0 tracce associate).

Regole per la gestione della coda

- La coda è intesa come un insieme di tracce e le playlist in coda vengono appiattite e aggiunto un riferimento alla playlist di appartenenza in modo da rendere più semplice il salto della playlist nell'ascolto shuffle.

Regole di Dominio sulla Riproduzione e Persistenza

- **Integrità Referenziale:** Se una traccia viene eliminata definitivamente dal Catalogo principale, il sistema deve garantire che il suo identificatore venga rimosso a cascata da tutte le Playlist che lo contenevano, mantenendo il file di salvataggio coerente.
- **Isolamento delle Operazioni (Undo):** Le modifiche allo stato della libreria o delle playlist (aggiunta/rimozione) mantengono in memoria lo stato precedente dell'azione per consentirne l'immediata revocabilità prima del consolidamento sul file di persistenza.

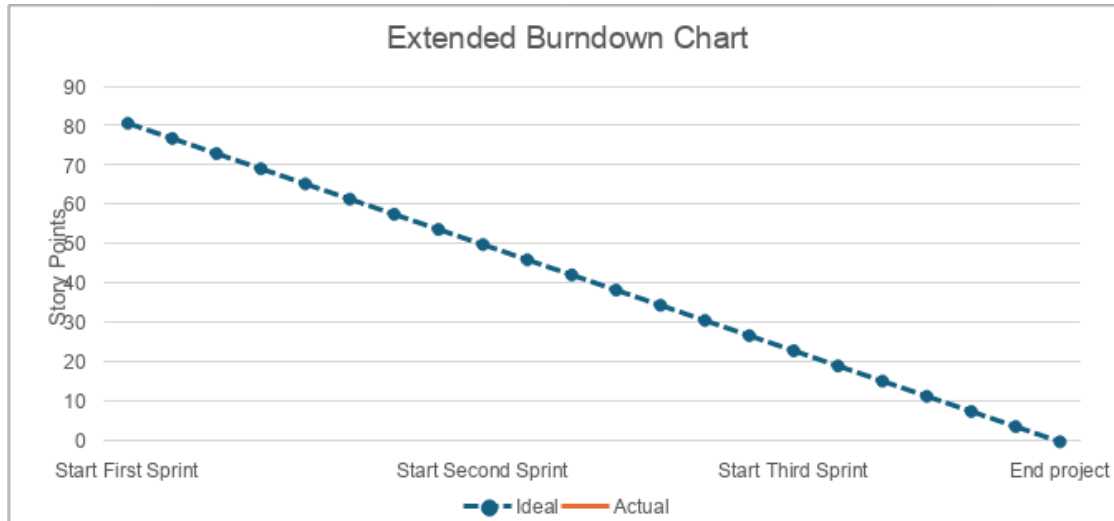
2.7. Definition of Done

Le Definition of Done saranno prese in considerazione per la validazione finale della completezza di ogni User Story.

- Tutti i Criteri di Accettazione specificati all'interno della User Story sono stati verificati e soddisfatti.
- Sono stati scritti e superati con successo gli Unit Test automatizzati (tramite framework JUnit) e i necessari test di integrazione a copertura della nuova logica di business.
- Il codice sorgente è pulito, privo di errori di compilazione e propriamente commentato, rispettando le convenzioni standard di Java (es. utilizzo di Javadoc per la documentazione di classi e metodi pubblici).
- Il codice è stato inserito nel repository GitHub del progetto ed è stato sottoposto a un processo di Peer-Review da parte di almeno un altro membro del team (tramite approvazione di una Pull Request) prima del merge definitivo.

2.8. Burndown Chart

La figura seguente illustra il Burndown Chart, che traccia l'andamento ideale del lavoro del team fino al termine previsto per la consegna. Il grafico è stato elaborato all'interno di un file Excel, che sarà presente nel repository su Github.



2.9. First Sprint Planning

In questo Planning, il team si è organizzato per incontrare gli obiettivi ideali definiti dalla stima di velocità calcolata. Inoltre, il team ha deciso di mantenere un numero di Story Points omogeneo anche per gli altri sprint, così da garantire una distribuzione equilibrata del carico di lavoro e una pianificazione più coerente durante lo sviluppo del progetto. Le User Stories selezionate per questo Sprint sono:

- **US1 – Creazione Tracce** (5 Story Points)
Funzionalità dedicata all'inserimento di nuove tracce musicali all'interno dell'archivio, con validazione dei campi e controllo dei duplicati.
- **US2 – Modifica Tracce** (3 Story Points)
Permette all'utente di modificare le informazioni delle tracce già presenti, mantenendo controlli di validità sui dati inseriti.
- **US3 – Eliminazione Tracce** (2 Story Points)
Consente la rimozione definitiva delle tracce dall'archivio tramite conferma dell'operazione.
- **US4 – Visualizzazione Tracce** (1 Story Point)
Funzionalità per visualizzare tutte le tracce presenti nell'archivio e i relativi metadati.
- **US6 – Creazione Playlist** (2 Story Points)
Permette all'utente di creare una nuova playlist identificata da un nome univoco, così da organizzare i brani presenti nell'archivio secondo specifici criteri.
- **US7 – Aggiunta Tracce a Playlist** (2 Story Points)
Consente di aggiungere tracce a una playlist, anche durante la riproduzione, evitando l'inserimento di duplicati.
- **US8 – Rimozione Tracce da Playlist** (1 Story Point)
Funzionalità dedicata alla rimozione di tracce da una playlist senza eliminarle dall'archivio musicale.
- **US9 – Visualizzazione Playlist** (1 Story Point)
Permette all'utente di visualizzare il contenuto di una playlist, mostrando nome, ordine delle tracce, titolo e autore.
- **US10 – Modifica Nome Playlist** (1 Story Point)
Consente di aggiornare il nome di una playlist per correggere errori o modificare le informazioni associate.
- **US16 – Riproduzione Traccia** (2 Story Points)
Consente all'utente di riprodurre una traccia selezionata dall'archivio musicale o da una playlist, avviando l'ascolto del brano scelto.
- **US17 – Riproduzione in Pausa** (2 Story Points)
Permette di mettere in pausa la riproduzione di una traccia e di riprenderla successivamente mantenendo il punto esatto di ascolto.
- **US23 – Aggiornamento dello Stato dell'Interfaccia Utente** (3 Story Points)
Funzionalità dedicata all'aggiornamento dinamico dell'interfaccia durante la riproduzione, mostrando in tempo reale l'avanzamento del brano senza necessità di ricaricare la pagina.

La scelta di queste User Story è giustificata dal fatto che, oltre ad essere coerente con la capacità del team, rappresentano le capacità di base che un'applicazione di gestione musicale dovrebbe garantire. Queste User Story rappresentano il flusso esecutivo naturale: dall'aggiunta di un brano fino alla sua organizzazione all'interno di playlist. Raggrupparle nello stesso Sprint consente di sviluppare e validare

in modo completo l'intero ciclo di utilizzo principale del sistema, dalla gestione dei contenuti musicali fino alla loro fruizione e organizzazione.